

path-based methods aim to answer single-hop queries by combining neural and symbolic techniques, utilizing path information for more robust reasoning.

Existing symbolic path ranking methods consider only the direct path information between two entities, neglecting the rich context information surrounding entities. This often leads to suboptimal solutions. To address this, PathCon [61] incorporates both relational context and relational paths in the reasoning process. Relational context refers to the k -hop neighboring relations of a given entity, while relational paths are the connections between two entities. PathCon encodes relational context using Relational Message Passing to aggregate k -hop content information around a predicate. For encoding relational paths, PathCon identifies all paths between entities h and t of length no greater than k , and then uses an RNN to learn the representation of each path. After learning both context and path information, PathCon combines them. It concatenates the final embeddings of entities h and t and processes them through a neural network to obtain the final context embedding. An attention mechanism aggregates the relational path information. The final output is a probability distribution, computed based on the combined context and path embeddings.

Unlike previous methods, DeepPath [68] uses reinforcement learning to predict missing links. DeepPath learns paths rather than relying solely on random walks, framing the path-finding process as a Markov decision process. It trains a reinforcement learning agent to discover paths, using these paths as horn clauses for knowledge graph reasoning. In DeepPath, the agent is represented by a neural network, and the answer-finding process is modeled as a Markov decision process. The states are defined as the concatenation of the current entity embedding and the distance between the target and the current entity in the embedding space, while the action space consists of all unique relation types in the knowledge graph.

3.2.4 Neural symbolic rule mining. For all previous symbolic rule mining methods, the focus is on mining Horn clauses. In GraIL [55], the authors propose using subgraphs for reasoning, based on the idea that useful rules are contained in the subgraph around the query. GraIL applies graph neural networks (GNNs) to subgraphs surrounding the candidate edge, hypothesizing that subgraph structure provides evidence for inferring relations between nodes. This GNN-based method avoids explicit rule induction while remaining expressive enough to capture logical rules. The reasoning process in GraIL comprises three steps: first, extracting a subgraph around the candidate edge; second, assigning structural labels to nodes based on their distances from the target nodes to encode graph substructure; and third, running a GNN on the extracted subgraph to capture the rules. The GNN in GraIL uses the traditional combine-and-aggregate paradigm, where each node aggregates information from its neighbors using an aggregate function. To distinguish different relation types in the knowledge graph, GraIL employs relation-specific attention to weigh information from different neighbors. During inference, given a triplet (u, r_t, v) , GraIL obtains the subgraph representation through average pooling of all latent node representations. It then concatenates this subgraph representation with the target nodes' latent representations and a

learned embedding of the target relation. These concatenated representations are passed through a linear layer to obtain the score. The model is trained using gradient descent.

3.2.5 Summary. Neural-symbolic reasoning combines the strengths of neural networks and symbolic reasoning to tackle the complexities of knowledge graphs. Traditional symbolic reasoning methods, like rule-based expert systems, employ predefined rules for inference, while path-based approaches like the Path Ranking Algorithm utilize random walks to infer relationships. Hybrid methodologies, such as PathCon, integrate relational context and path information through neural networks, enhancing reasoning capabilities. Embedding techniques, including geometric and probabilistic embeddings, represent entities and relationships in continuous vector spaces, facilitating more flexible knowledge graph operations. Reinforcement learning-based methods, like DeepPath, utilize trained agents to navigate knowledge graphs and predict missing links. By merging neural and symbolic techniques, neural-symbolic reasoning offers a comprehensive approach to understanding and reasoning over complex knowledge graph structures, promising advancements in various applications requiring automated reasoning and inference.

4 REASONING FOR COMPLEX LOGIC QUERY

In this section, we generalize the query into logically more complex forms [37] and explain how to solve them using neural and symbolic methods. Compared to simple-hop queries, the additional complexity is introduced by involving more “elements” in logical language, such as multiple predicates, quantifiers, and variables. The fundamental motivation of complex queries also follows the narrative of single-hop query prediction, where we want to derive new knowledge but with more logical constraints. We also refer readers to the earlier survey [66] in this direction. Both single-hop queries and multi-hop queries fall under the broader category of complex queries. However, to differentiate knowledge graph completion from complex logical query answering, we treat single-hop queries and complex logical queries as two distinct components.

4.1 What is Complex Query?

General logical queries follow the definitions of mathematical logic and model theory [37]. The previous but perhaps not up-to-date survey provides more rigorous definitions and discussions [66]. Regarding logical language coverage, complex queries studied on knowledge graphs are still preliminary compared to parallel studies in databases [29] and semantic web research [38].

In the literature, a general term describing complex queries on knowledge graphs is the general “first-order query” or “first-order logical query” [46, 47]. However, recent rigorous characterization [76] distinguished the queries discussed in the definition, and queries studied in empirical methods and benchmarks are two overlapping query families. The first is existential first-order queries that appeared in definitions of many works [33, 47], and the second is the tree-formed queries widely adopted in the empirical construction of benchmarks [65]. Most empirical results remain credible despite the fine-grained differences in query families.

We hereby introduce the definitions of two queries based on a fragment of the first-order language. A term is either an entity $e \in V$ or a variable. A variable is a non-determined entity whose value can

be taken from V . A variable can be either existentially quantified or not. Universally quantified variables are not considered yet in the literature. An atomic formula is $r(a, b)$ where $r \in E$ is the relation. Then, we define the Existential First-Order (EFO) formulae.

DEFINITION 3 (EXISTENTIAL FIRST ORDER FORMULA). *The set of the existential formulas is the smallest set Φ that satisfies the following property:*

- (i) *If atomic formula $r(a, b)$, itself and its negation $r(a, b), \neg r(a, b) \in \Phi$, where a, b are either entities in V or variables, r is the relation in E .*
- (ii) *If $\phi, \psi \in \Phi$, then $(\phi \wedge \psi), (\phi \vee \psi) \in \Phi$*
- (iii) *If $\phi \in \Phi$ and x_i is any variable, then $\exists x_i \phi \in \Phi$.*

When there is more than one free variable, an EFO formula is an EFO query. In most previous studies, only one existential variable is considered, leading to the family of EFO-1, denoted as Φ . The families with more than one variable are titled EFO-k [75]; so far, there is no specific method targeting EFO-k. The key feature of EFO queries is that the logical negation is only attached to atomic formulas, defined by rule (i). Consequently, one can always move existential quantifiers at the beginning of the formula as the prenex normal form [37]. Moreover, it is always convenient to reorganize the logical conjunctions and disjunctions into either Disjunctive Normal Form (DNF) or Conjunctive Normal Form (CNF). One common way to define the EFO-1 query is by the DNF and the conjunctive queries.

Specifically, the queries q can be written as follows.

DEFINITION 4 (EFO-1 QUERY). *An EFO-1 query is defined as*

$$q(y) = \exists x_1, \dots, x_k, \pi_1(y) \vee \dots \vee \pi_n(y), \quad (1)$$

where y is the variable to be queried, $x_1, \dots, x_k$ are existentially quantified variables, and $\pi_n(y)$ is the conjunctive query to be defined in the following parts.

DEFINITION 5 (CONJUNCTIVE QUERY). *A conjunctive logical query is written as*

$$\pi_i(y) = \exists x_1, \dots, x_k : \varrho_1^i \wedge \varrho_2^i \wedge \dots \wedge \varrho_m^i,$$

where each ϱ_j^i is the atomic formula $r(a, b)$ or its negation $\neg r(a, b)$.

Another query family that is well studied is formally defined as the Tree-Formed (TF) queries Φ_{tf} .

DEFINITION 6 (TREE-FORM QUERY). *The set of the Tree-Form queries is the smallest set Φ_{tf} such that:*

- (i) *If $\phi(y) = r(a, y)$, where $a \in E$, $r \in R$, then $\phi(y) \in \Phi_{\text{tf}}$;*
- (ii) *If $\phi(y) \in \Phi_{\text{tf}}$, $\neg \phi(y) \in \Phi_{\text{tf}}$;*
- (iii) *If $\phi(y), \psi(y) \in \Phi_{\text{tf}}$, then $(\phi \wedge \psi)(y) \in \Phi_{\text{tf}}$, $(\phi \vee \psi)(y) \in \Phi_{\text{tf}}$;*
- (iv) *If $\phi(y) \in \Phi_{\text{tf}}$ and y' is any variable, then $\psi(y') = \exists y. r(y, y') \wedge \phi(y) \in \Phi_{\text{tf}}$.*

One key feature of Φ_{tf} is that the answer set can be constructed recursively through set operations, such as union, intersection, and complement. As specifically shown in Definition 6, rule (ii) corresponds to the set complement against an implicit universe set; rule (iii) relates to the set intersection and union to logical conjunction and disjunction; and rule (iv) for set projection. Under the context

of tree-form queries, we use logical connectives (conjunction, disjunction, and negation) and set operations (intersection, union, and complement) interchangeably.

EFO-1 and tree-form query families are different but not mutually exclusive ($\text{EFO-1} \cap \text{TF} \neq \emptyset$). There are also queries in the tree-form family but not in EFO-1 and vice versa. Detailed discussions of query syntax can be found in [76]. The follow-up part then explains neural and neural-symbolic methods for TF and EFO-1 queries.

4.2 Neural Methods

Neural methods conduct logical reasoning in a fixed-dimensional embedding space, where previous insights from knowledge graph embeddings can be applied. The methods for tree-formed queries and EFO-1 queries differ significantly in two ways. In short, methods targeting tree-formed queries emphasize the modeling of set operations [65]. Methods for the EFO-1 query stress the DNF formulation and the conjunctive query.

4.2.1 Tree-form query. The first attempt to solve a tree-form query begins with the logical conjunction, or more specifically; the final answer set can be derived by set projection and set intersection. As the earliest example, GQE [21] embeds graph nodes in a low-dimensional space and represents set projection and intersection as neural operations in the embedding space. Consequently, terms in the query, including constant entities, existential variables, and free variables, can be represented or computed as the embedding. Then, the nearest neighbor search is used to find answers. The embeddings and neural models are trained on synthetic datasets by an end-to-end auto-differentiation. Follow-up methods followed the key design principles: (1) represent the terms into low-dimensional embeddings; (2) set operations are modeled by differentiable operations in the embedding spaces; (3) identify the final answers by nearest neighbor search. Moreover, the supported set operations are expanded to set union (logical disjunction) and set complement (logical negation).

Notably, the set intersection, union, and negation provide some natural properties and intuitions. An example is the box-embedding space and various query embedding methods. Query2Box [46] proposes to model queries as boxes (i.e., hyper-rectangles), where a set of points inside the box corresponds to a set of answer entities of the query. Set intersections can be naturally represented as geometric intersections of boxes in the space. On the other hand, the set union cannot be modeled by the geometric union of boxes because the resulting shape is not a box. This issue can be indirectly addressed by transforming queries into a DNF. Furthermore, NewLook [33] adopts a neural network to relearn the box embedding at each projection operation to mitigate the cascading error problem and also introduces a new logic operator, *set difference*, so that the set complement can be modeled by the equivalently. Besides the box-embedding space, other kinds of embedding spaces are also widely explored, including the space of convex cones [82], parametric probabilistic distributions [47, 71], and vectors [9, 63].

4.2.2 EFO-1 query. This part only focuses on methods that are capable of solving queries that are EFO-1 but not tree-form. Such queries are characterized by the query graph of the sub-conjunctive query. Specifically, such a query can be represented as a simple

(with multi-edges) or cyclic graph, which prohibits the perspective that regards the logical query as compositional operations [76]. Instead, a more natural framework is to consider the atomic formulas or their negation in the conjunctive query as constraints in constraint satisfaction problems. One practical approach is to adopt the graph neural networks on the query graph. LMPNN injects the logical constraint in the message and connects the complex query to message-passing networks [64]. More sophisticated neural architectures like transformers are investigated on the query graph [69] and messages [78].

4.3 Neural-Symbolic Methods

The neural-symbolic methods for complex query answering integrate symbolic algorithms, which have been extensively studied in the database community. The neural part of such approaches, on the other hand, is less capable than that of neural approaches. In practice, the neural part of neural symbolic approaches is mainly chosen as link predictors or knowledge graph embeddings. Different from the previous discussions on the neural approach, neural symbolic approaches rely heavily on the symbolic algorithm; thus, they can solve a more extensive set of queries.

Almost all symbolic algorithms search for a proper assignment of variables $x_1 = e_1, \dots, x_k = e_k$, and $y = a$ to satisfy the logical constraints in queries. Combined with neural link predictors, the boolean value of satisfaction is turned into a continuous score or normalized into $[0, 1]$ as fuzzy truth values. The preliminary approach models the adjacent matrices of KG for each relation, with elements as the fuzzy truth value of triples. The details of how to normalize the output of the link predictor into fuzzy truth values vary in different methods. However, it does not change the nature of the problem as a search process. Several search strategies can be seamlessly applied to such a problem. For example, beam search realized for the acyclic query graph is known as CQD [3], and search on acyclic query graph with additional backtracking is proposed as QTO [5]. The generalization from acyclic to cyclic and multi-edge query graphs is known as FIT [76]. Many algorithmic results are also available to accelerate the algorithm, such as using a count-min sketch in EMQL [51] to store the entity set for each query node and using vector similarity to find similar results during the search process.

Neural link predictors can be deeply engaged with search and not just materialized as adjacency matrices. CQD-CO relaxed the search problem from symbolic assignment spaces into neural embedding space, thus enabling gradient-based optimization with differentiable link predictions [3]. CQD-A, as a more advanced method, can adapt knowledge graph embeddings from the feedback of the training data [4]. A similar but technically different approach is the GNN-QE, where the link predictor is not just embeddings but a graph neural network to be learned from the feedback of the search results [84].

Recently, some approaches have attempted to combine large language models (LLMs) with neural-symbolic methods to address this problem. For instance, in [36], a framework is proposed that decomposes complex questions into multiple subquestions, which are then individually answered by LLMs. Simultaneously, neural-symbolic methods are applied to incomplete knowledge graphs. At

each time step, the results from the LLMs and knowledge graph reasoning are integrated to produce a cohesive response.

5 REASONING FOR NATURAL LANGUAGE QUERY

5.1 Reasoning for Single-turn Query

When the input query is a natural language sentence, existing methods can be divided into several categories, such as semantic parsing-based methods, information retrieval-based methods, and embedding-based methods.

For example, PullNet [52] is information retrieval-based methods that retrieve a subgraph of candidate answers from the knowledge base to guide prediction. KV-Mem [39] and EmbedKGQA [49] are embedding and deep learning-based methods that use deep learning networks to embed the question into a point in the embedding space and find answers according to a similarity function. In PrefNet [32], a reranking based method is used to rerank all candidate answers to get better results. In semantic parsing-based methods, a general strategy to answer the question is to transform the question into a query graph and search for the answer according to the query graph. For example, in [74], Xi et al. propose a model with candidate query graphs ranking and true query graph generation components. By iteratively updating these components, their performance improves. The query graph generated can then be used to search the KG. In [34], Liu et al. propose a multi-task model to tackle KGQA and KGC simultaneously. They transform the multi-hop query into a path in the knowledge graph and use it to complete the knowledge graph, jointly training the model for both tasks.

Recently, reinforcement learning-based methods have been used to answer natural language questions on the knowledge graph. Zhang et al. [79] use a KG as the environment and propose an RL-based agent model to navigate the KG to find answers to input questions. Similarly, in [12, 30, 68], authors use RL models to find paths in the KG for answering input queries. Other studies, such as [2, 40, 45, 56, 83], integrate RL with other methods to create more human-like systems.

5.2 Reasoning for Multi-turn Query

Various approaches have been used to reason for multi-turn questions. For instance, Conquer [26] notices that whenever there is a reformulation, it is likely that the previous answer was wrong, and when there is a new intent, it's more likely that the previous answer was correct. Based on this idea, Conquer treats reformulations as implicit feedback to train a reinforcement learning model. The idea is to position multiple reinforcement learning agents on relevant entities, allowing these agents to walk over the knowledge graph to answer in its neighborhood.

Despite the common use of reinforcement learning in conversational question answering, it has many disadvantages. For example, the paths in the knowledge graph found by agents are often very similar, making them hard to distinguish. In this example, all these five paths lead to the answer entity, but they have different intermediate nodes. Besides, the reward is sparse, making the model hard to train. To solve these problems, PRALINE [25] uses a contrastive representation learning approach to rank KG paths for retrieving the correct answers effectively. Extracted KG paths leading to correct